

Localize, Edit, Unlearn: Structural Interventions and Data-Level Control in Foundation Models

Keivan Rezaei



Ph.D. Thesis Defense

September 17, 2026

Committee: Prof. Soheil Feizi (Chair) • Prof. Mohammadtaghi Hajiaghayi (Co-Chair) • Prof. Sarah Wiegrefe • Prof. David Jacobs • Prof. Maria K. Cameron

Table of Content

- Introduction
- Localize
- Edit
- Unlearn
- Conclusion
- References

Research Theme

Knowledge Localization
& Model Editing

Data Interpretability &
Machine Unlearning

Generative AI: 2022 – Now

Text-to-Image Models

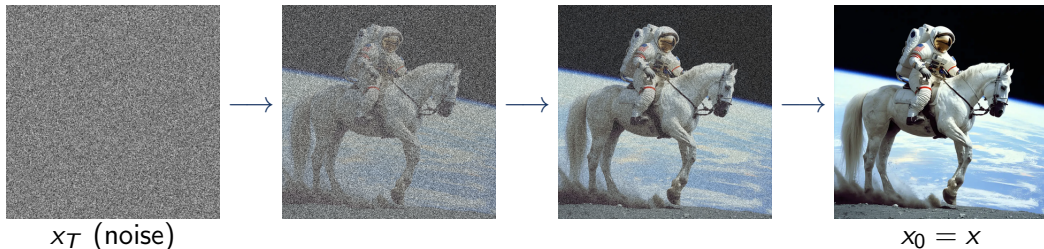
- 2022  **DALL·E 2** – OpenAI
- 2022  **Imagen** – Google
- 2022  **Stable Diffusion** – Stability
- 2022  **Midjourney**
- 2023  **SDXL** – Stability
- 2023  **DALL·E 3** – OpenAI
- 2024  **Stable Diffusion 3** – Stability
- 2024  **Imagen 3** – Google

Large Language Models

- 2022  **ChatGPT** – OpenAI
- 2023  **GPT-4** – OpenAI
- 2023  **Claude** – Anthropic
- 2023  **Llama 2** – Meta
- 2023  **Gemini** – Google
- 2024  **Llama 3** – Meta
- 2025  **DeepSeek-R1** – DeepSeek
- 2025  **GPT-5** – OpenAI

Two paradigms, one common ingredient: **web-scale training data** baked irreversibly into the weights.

Text-to-Image: Diffusion



"an astronaut riding a horse in space" = c



$$\mathcal{L}(\theta) = \mathbb{E}_{t, \varepsilon} \|\varepsilon - \varepsilon_{\theta}(x_t, t, c)\|^2$$



U-Net $\rightarrow$ Transformer

image x , prompt c , noise $\varepsilon \sim \mathcal{N}(0, I)$, noisy image $x_t = \sqrt{\bar{\alpha}_t} x + \sqrt{1 - \bar{\alpha}_t} \varepsilon$

Large Language Models: Next-Token Prediction

The capital of France is __




$$\mathcal{L}(\theta) = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_1, \dots, x_{t-1})$$
 Transformer

tokens $x_1, x_2, \dots, x_T$ of a training sequence

Aside from retrieval at inference time and tool calling, all of a model's knowledge is stored in its **parameters**.

Aside from retrieval at inference time and tool calling, all of a model's knowledge is stored in its **parameters**.

We aim to understand AI from a **model** and **data** perspective.

Research Theme

Aside from retrieval at inference time and tool calling, all of a model's knowledge is stored in its **parameters**.

We aim to understand AI from a **model** and **data** perspective.

Understanding Model

Where is the knowledge?
Can we localize it?

Understanding Data

How does a training sample
affect the model?
Can we measure that?

Research Theme

Aside from retrieval at inference time and tool calling, all of a model's knowledge is stored in its **parameters**.

We aim to understand AI from a **model** and **data** perspective.

Understanding Model

Where is the knowledge?
Can we localize it?



Understanding Data

How does a training sample
affect the model?
Can we measure that?



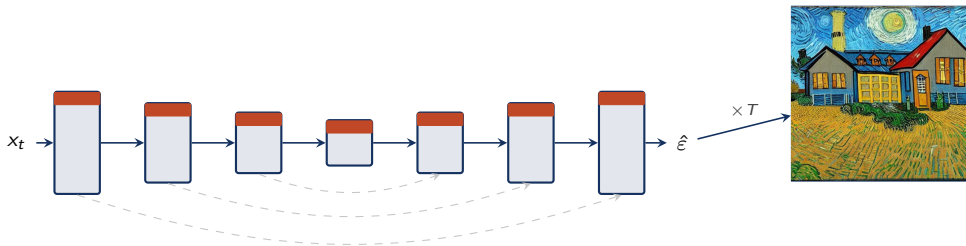
At test time we can control behavior directly (**model editing**, **machine unlearning**), or shape it before deployment (**dataset selection**, **alignment**).

Research Theme

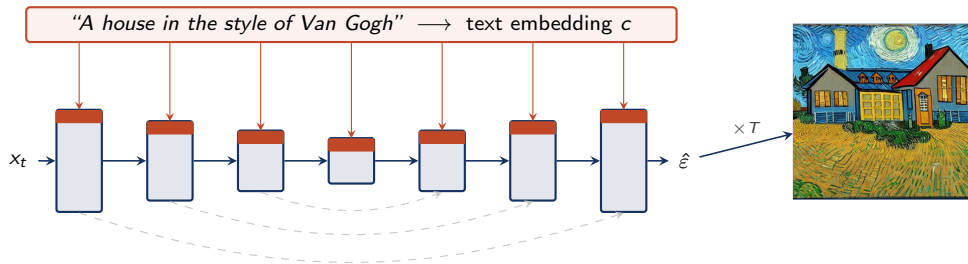
**Knowledge
Localization & Model
Editing**

Data Interpretability &
Machine Unlearning

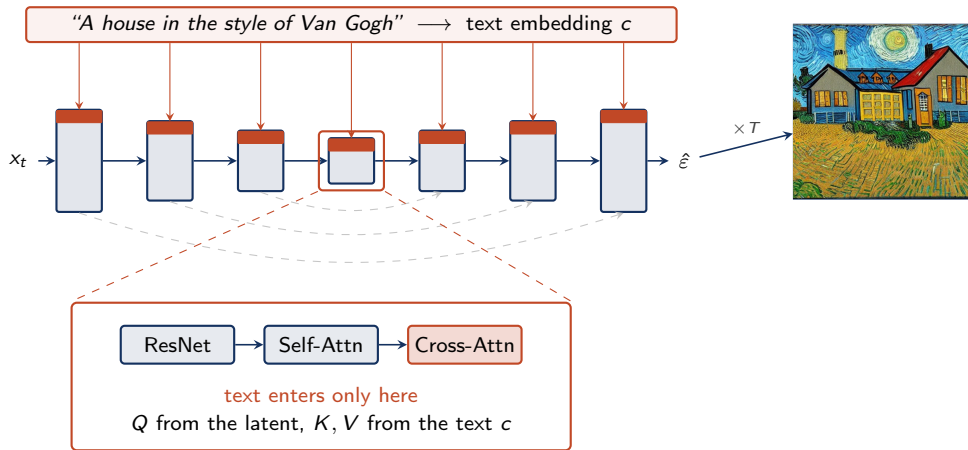
Inside a U-Net Diffusion Model: Stable Diffusion 2 (2022)



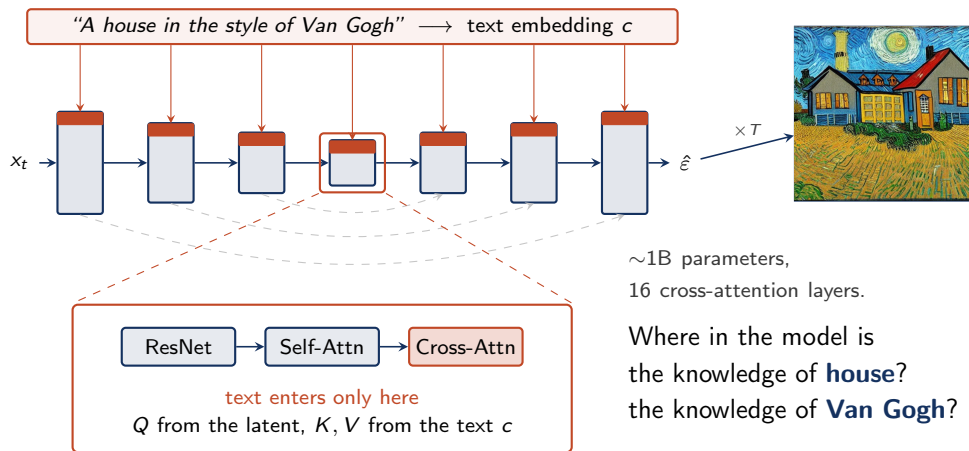
Inside a U-Net Diffusion Model: Stable Diffusion 2 (2022)



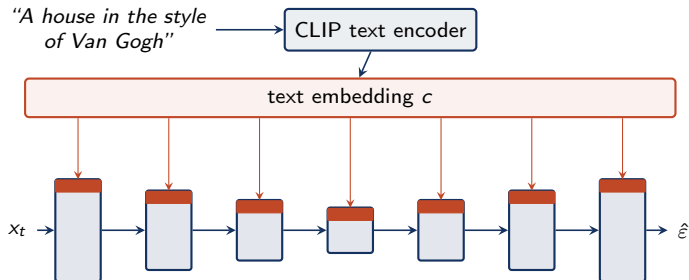
Inside a U-Net Diffusion Model: Stable Diffusion 2 (2022)



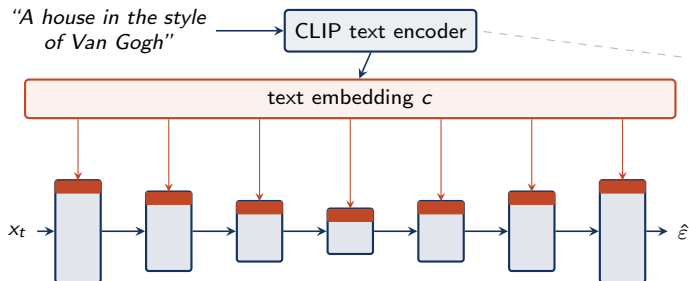
Inside a U-Net Diffusion Model: Stable Diffusion 2 (2022)



Are All Layers Equal?

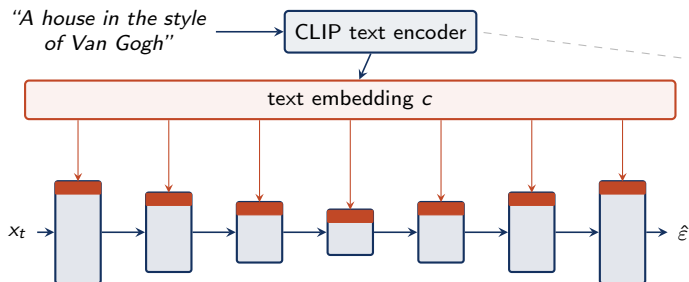


Are All Layers Equal?



Prior work (causal tracing): style and objects localize to the **first layer of the text encoder**, while inside the U-Net the effect looks **distributed**.

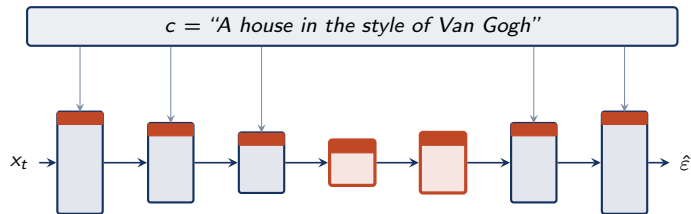
Are All Layers Equal?



Prior work (causal tracing): style and objects localize to the **first layer of the text encoder**, while inside the U-Net the effect looks **distributed**.

All **16 cross-attention layers** receive the *same* c .
Are all of them equally responsible?

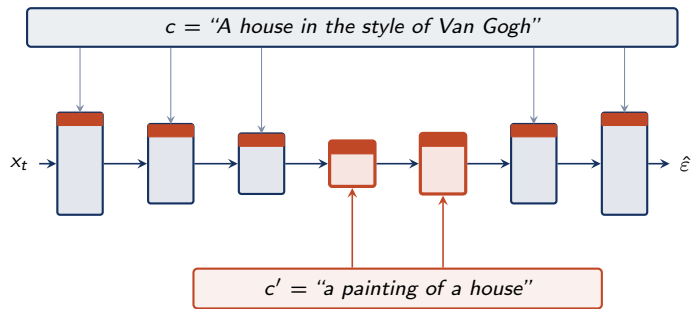
LocoGen: Give a Subset a Different Prompt



all layers get c
style present

Basu, Rezaei, et al. *On Mechanistic Knowledge Localization in Text-to-Image Generative Models*. ICML 2024.

LocoGen: Give a Subset a Different Prompt



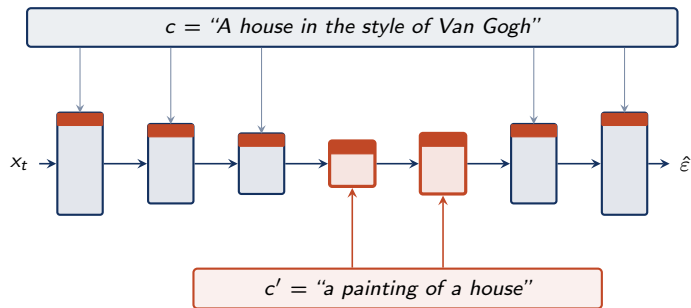
all layers get c
style present



subset gets c'
style ablated

Basu, Rezaei, et al. *On Mechanistic Knowledge Localization in Text-to-Image Generative Models*. ICML 2024.

LocoGen: Give a Subset a Different Prompt



all layers get c
style present



subset gets c'
style ablated

The style disappears when only **a few adjacent layers** see c' .

Is that where "Van Gogh" is stored?

Basu, Rezaei, et al. *On Mechanistic Knowledge Localization in Text-to-Image Generative Models*. ICML 2024.

LocoGen: Searching for the Controlling Layers

Which m of the M cross-attention layers should get c' ?

Brute force tries all $\binom{M}{m}$ subsets (SD-XL: 12M).

LocoGen: Searching for the Controlling Layers

Which m of the M cross-attention layers should get c' ?

Brute force tries all $\binom{M}{m}$ subsets (SD-XL: 12M).

Restricting the search to **adjacent** layers leaves only $M - m + 1$ windows (SD-XL: 66).

LocoGen: Searching for the Controlling Layers

Which m of the M cross-attention layers should get c' ?

Brute force tries all $\binom{M}{m}$ subsets (SD-XL: 12M).

Restricting the search to **adjacent** layers leaves only $M - m + 1$ windows (SD-XL: 66).

LocoGen

```
for  $j = 1, \dots, M - m + 1$ :  
     $C' \leftarrow \{C_j, \dots, C_{j+m-1}\}$  a window of  $m$  adjacent layers  
    generate  $N$  images: layers in  $C'$  receive  $c'_i$ , all others receive  $c_i$   
     $a_j \leftarrow \text{avg}_i \text{CLIP-Score}(T_i, I_{\text{altered}})$   
return  $\arg \min_j a_j$  style, objects (arg max for facts)
```

LocoGen: Searching for the Controlling Layers

Which m of the M cross-attention layers should get c' ?

Brute force tries all $\binom{M}{m}$ subsets (SD-XL: 12M).

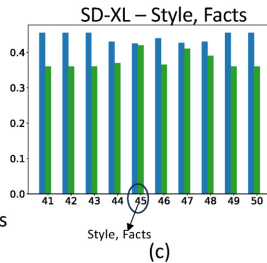
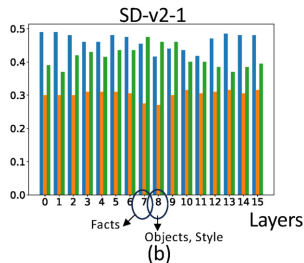
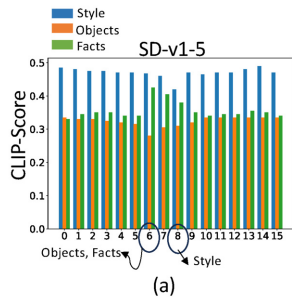
Restricting the search to **adjacent** layers leaves only $M - m + 1$ windows (SD-XL: 66).

LocoGen

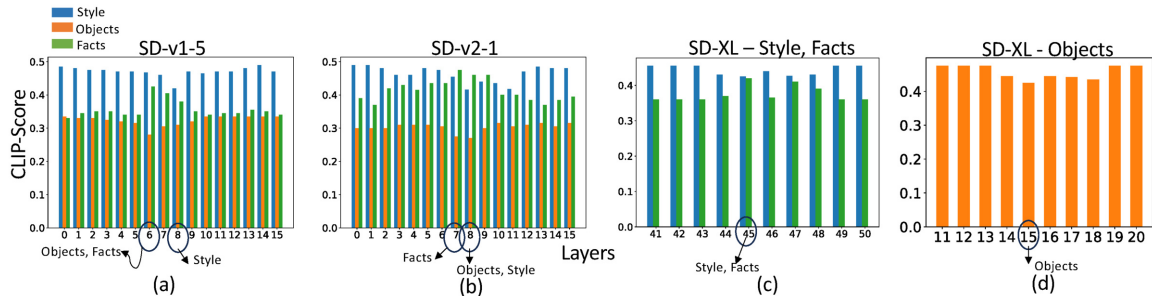
```
for  $j = 1, \dots, M - m + 1$ :  
     $C' \leftarrow \{C_j, \dots, C_{j+m-1}\}$  a window of  $m$  adjacent layers  
    generate  $N$  images: layers in  $C'$  receive  $c'_i$ , all others receive  $c_i$   
     $a_j \leftarrow \text{avg}_i \text{CLIP-Score}(T_i, I_{\text{altered}})$   
return  $\arg \min_j a_j$  style, objects (arg max for facts)
```

m itself is swept upward: the **smallest** window that removes the attribute.

Reading the Location Off a CLIP-Score Sweep



Reading the Location Off a CLIP-Score Sweep



Each bar is one candidate window.

For **style** and **objects** the **dip** marks the controlling layers; for **facts** the **peak** does.

How Few Layers Are Enough?

Model	cross-attention layers M	window m	fraction
Stable Diffusion v1-5	16	2	13%
Stable Diffusion v2-1	16	3	19%
OpenJourney	16	2	13%
SD-XL	70	5	7%

Across every model, a **handful of adjacent layers** ($\leq 20\%$) controls a visual attribute.

The window is reported by its start l : for SD-v1-5 ($m = 2$), “layer 8” means layers 8–9.

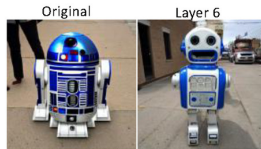
The Same Recipe for Style, Objects and Facts



Original Prompt: 'A house in the style of Van Gogh'



'A town in the style of Monet'



'R2D2 in street'



'President of United States'

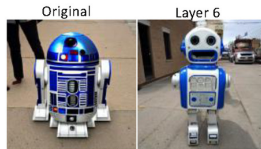
The Same Recipe for Style, Objects and Facts



Original Prompt: 'A house in the style of Van Gogh'



'A town in the style of Monet'



'R2D2 in street'



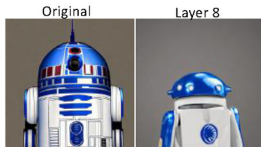
'President of United States'



Original Prompt: 'A house in the style of Van Gogh'



'A town in the style of Monet'

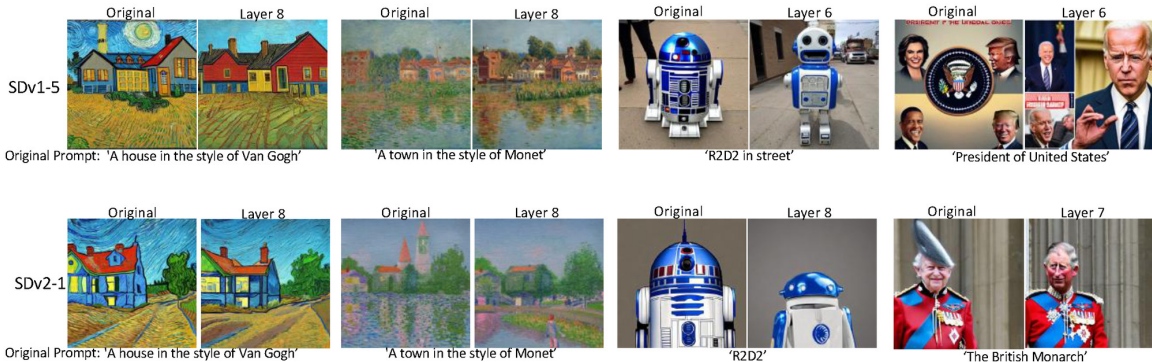


'R2D2'



'The British Monarch'

The Same Recipe for Style, Objects and Facts



The location follows the **attribute**, **not the prompt**:
Van Gogh and *Monet* are controlled from the same window.

We Can Localize. What Now?

Localize

a few adjacent cross-attention
layers carry the concept



We Can Localize. What Now?

Localize

a few adjacent cross-attention
layers carry the concept



Edit

write the change into those
weights, once and for all

We Can Localize. What Now?

Localize

a few adjacent cross-attention
layers carry the concept



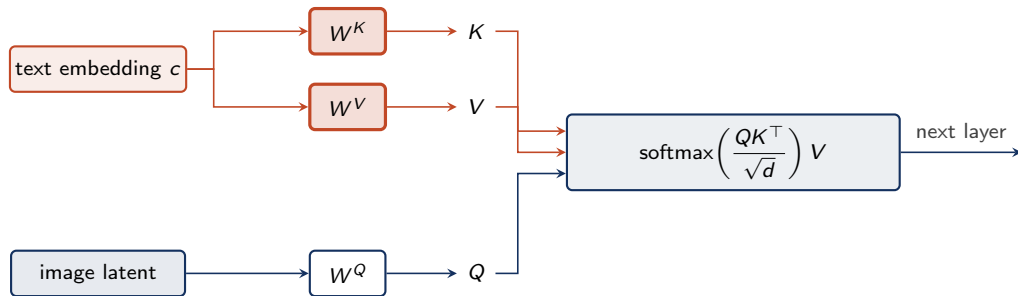
Edit

write the change into those
weights, once and for all

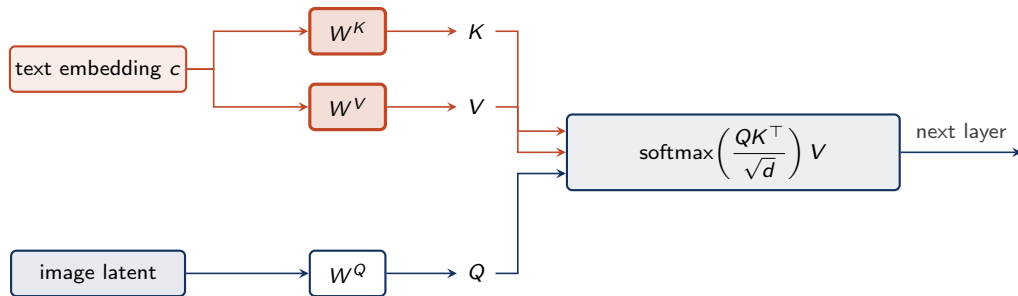
Swapping the input only helps **at inference**, one prompt at a time.

Can we put the change in the weights instead?

Inside Cross-Attention



Inside Cross-Attention



Only W^K and W^V ever see the text.

LocoEdit rewrites them, in closed form, at the located layers.

LocoEdit: Rewrite the Located Layers

N **prompt pairs** drawn from the whole attribute.

C_{orig}

"A house in the style of Van Gogh"

"A starry night in the style of Van Gogh"

⋮

C_{target}

"a painting of a house"

"a painting of a starry night"

⋮

LocoEdit: Rewrite the Located Layers

N **prompt pairs** drawn from the whole attribute.

C_{orig}

"A house in the style of Van Gogh"
"A starry night in the style of Van Gogh"
⋮

C_{target}

"a painting of a house"
"a painting of a starry night"
⋮

$$\min_{W^K} \|C_{\text{orig}} W^K - C_{\text{target}} \hat{W}^K\|_2^2 + \lambda_K \|W^K - \hat{W}^K\|_2^2$$

LocoEdit: Rewrite the Located Layers

N **prompt pairs** drawn from the whole attribute.

C_{orig}

"A house in the style of Van Gogh"
"A starry night in the style of Van Gogh"
 $\vdots$

C_{target}

"a painting of a house"
"a painting of a starry night"
 $\vdots$

$$\min_{W^K} \|C_{\text{orig}} W^K - C_{\text{target}} \hat{W}^K\|_2^2 + \lambda_K \|W^K - \hat{W}^K\|_2^2$$

$$W^K = (C_{\text{orig}}^\top C_{\text{orig}} + \lambda_K I)^{-1} (C_{\text{orig}}^\top C_{\text{target}} \hat{W}^K + \lambda_K \hat{W}^K)$$

one linear solve: no gradients, no fine-tuning; same for W^V .

How Little Gets Touched

SD-v1-5, *style*: layers 8–9, each with $W^K, W^V \in \mathbb{R}^{768 \times 1280}$.

2 layers $\times$ 2 matrices $\times$ 0.98M $\approx$ **4M parameters**
out of a U-Net of $\approx$ **860M**
that is $\approx$ **0.5%**

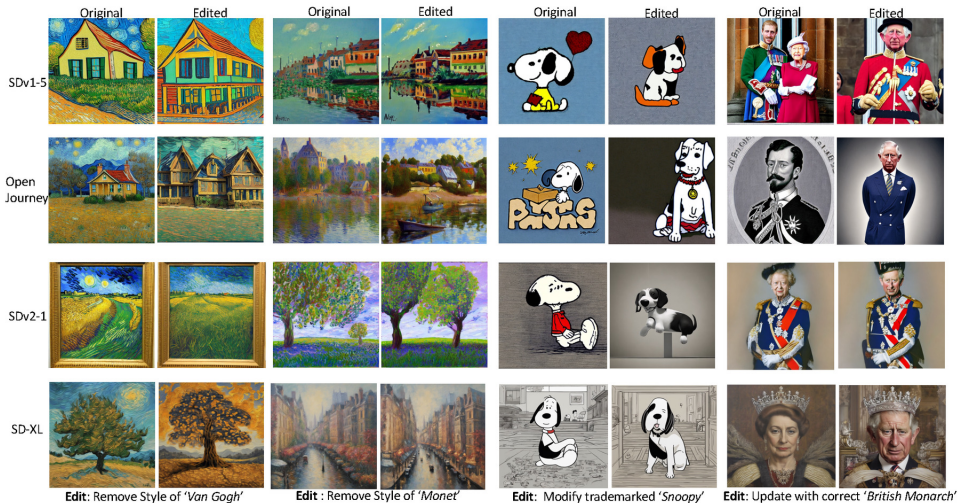
How Little Gets Touched

SD-v1-5, *style*: layers 8–9, each with $W^K, W^V \in \mathbb{R}^{768 \times 1280}$.

2 layers $\times$ 2 matrices $\times$ 0.98M $\approx$ **4M parameters**
out of a U-Net of $\approx$ **860M**
that is $\approx$ **0.5%**

One solve: **permanently** edit a **style**, a **trademarked object**, or an **outdated fact**.

LocoEdit: Edits Across Models and Attributes



The Edit Leaves Everything Else Alone

320 prompts from MS-COCO, unrelated to the edit.

Model	style-edited model	object-edited model
SD-v1.5	30.04	30.04
LocoEdit	29.99	29.40

Table: CLIP-Score of generations for the generic prompts.

[placeholder]

Two Loose Ends

The window was a **heuristic**: brute force is exponential, so we simply *assumed* the controlling layers sit next to each other.

Two Loose Ends

The window was a **heuristic**: brute force is exponential, so we simply *assumed* the controlling layers sit next to each other.

And text-to-image has moved off U-Nets: **PixArt- α** , **FLUX**, **SANA** are **Diffusion Transformers**, where “a window of adjacent layers” means little.

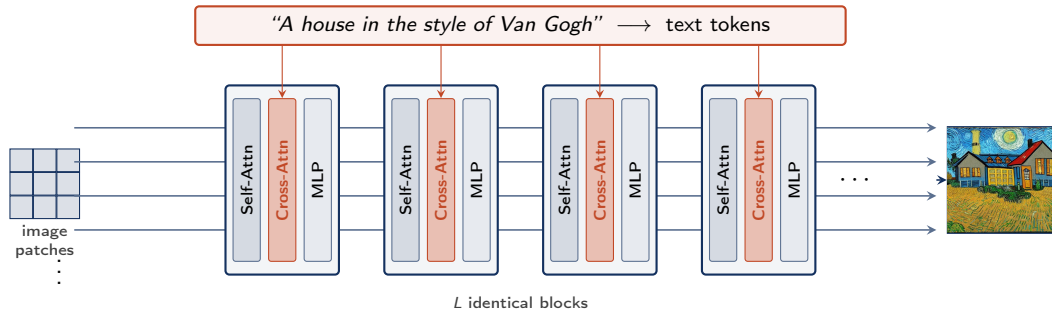
Two Loose Ends

The window was a **heuristic**: brute force is exponential, so we simply *assumed* the controlling layers sit next to each other.

And text-to-image has moved off U-Nets: **PixArt- α** , **FLUX**, **SANA** are **Diffusion Transformers**, where “a window of adjacent layers” means little.

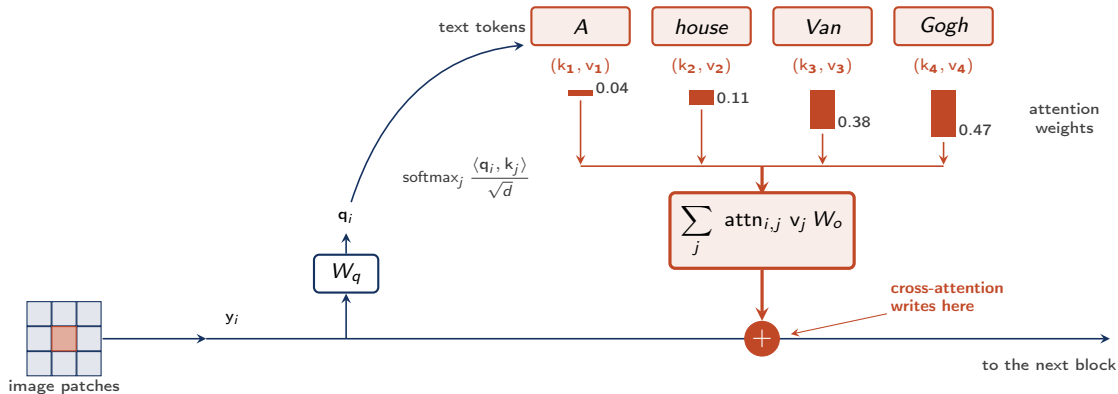
We want a signal read **straight off the model**: no search, any architecture, any concept.

Inside a Diffusion Transformer



Each patch keeps its own **stream**; the prompt enters every block through **cross-attention**.

What Cross-Attention Writes



Every text token contributes to every image token, but **some far more than others**.

Localizing by Attention Contribution

How hard does text token x_j **push** on image token y_i in **this block**?

$$\text{cont}_{i,j} = \|\text{attn}_{i,j} v_j W_o\|_2$$

Localizing by Attention Contribution

How hard does text token x_j **push** on image token y_i in **this block**?

$$\text{cont}_{i,j} = \left\| \sum_{h=1}^H \text{attn}_{i,j}^h v_j^h W_o^h \right\|_2$$

Attention runs H heads in parallel, so add their writes first.

Localizing by Attention Contribution

How hard does text token x_j **push** on image token y_i in **this block**?

$$\text{cont}_{i,j} = \left\| \sum_{h=1}^H \text{attn}_{i,j}^h v_j^h W_o^h \right\|_2$$

Attention runs H heads in parallel, so add their writes first.

Summing over image tokens i gives how much x_j writes into the **whole image** at this layer.

Localizing by Attention Contribution

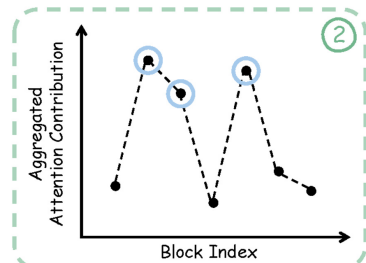
How hard does text token x_j **push** on image token y_i in **this block**?

$$\text{cont}_{i,j} = \left\| \sum_{h=1}^H \text{attn}_{i,j}^h v_j^h W_o^h \right\|_2$$

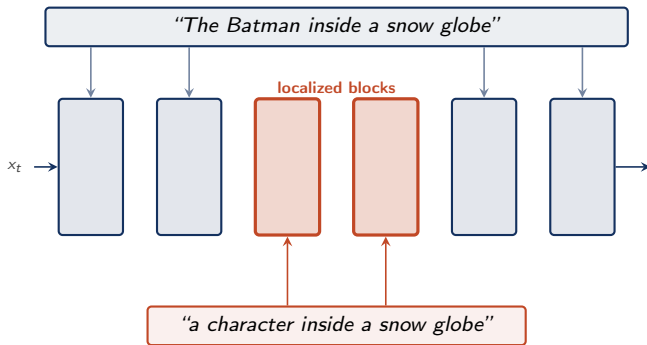
Attention runs H heads in parallel, so add their writes first.

Summing over image tokens i gives how much x_j writes into the **whole image** at this layer.

For a meaningful x_j (an artist, an object, a character), rank the layers by that score and keep the **top contributors**.



Does It Work?



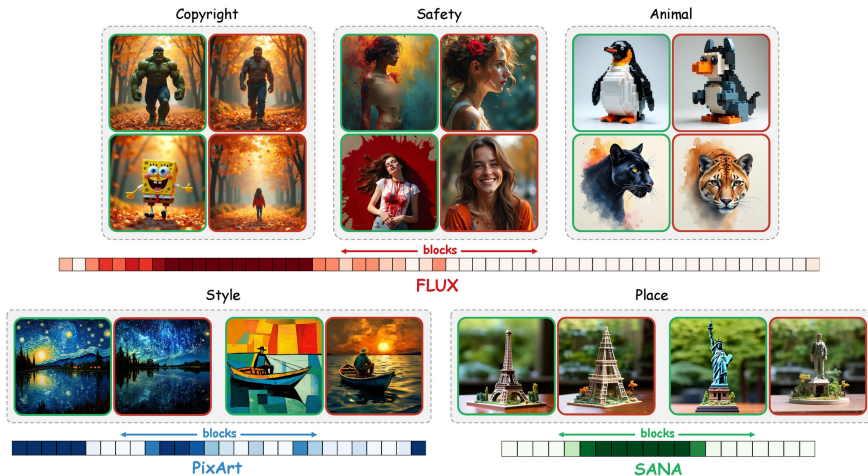
every block gets
the prompt



Batman is gone

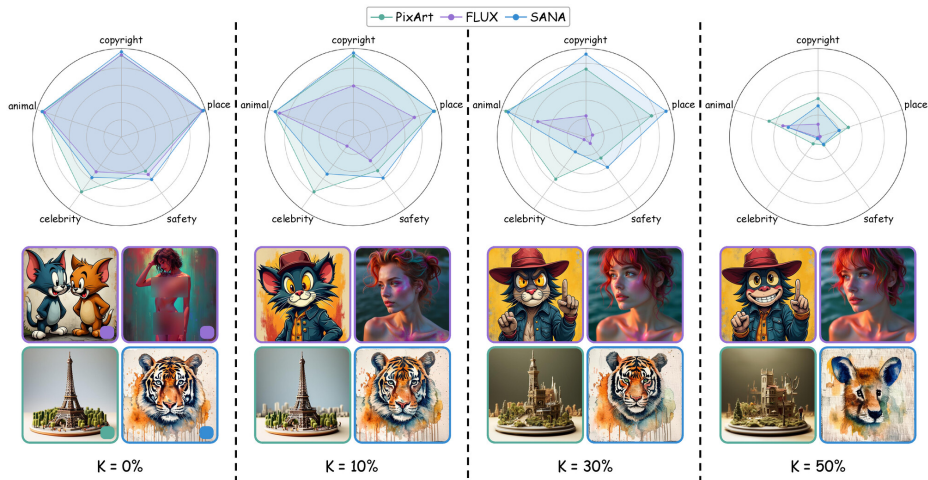
Hand the localized blocks a **knowledge-agnostic twin** of the prompt. If the concept vanishes, those blocks were carrying it.

Where Knowledge Lives, Model by Model



Green: normal generation. Red: knowledge withheld from the localized blocks. Bars: how often each block is picked.

Some Concepts Are Tighter Than Others



Intervening on the top- $K\%$ of blocks. Copyright, place and celebrity collapse early; animals are spread much wider.

Even Two Artists Need Different Blocks

In PixArt- α , out of **28 blocks**:

Caulfield $K = 4$

Picasso $K = 8$

Monet $K = 10$

Van Gogh $K = 12$

Just $K = 4$ already pins down the style of $\sim 80\%$ of the artists.



"a farmer working in the field in the style of Pablo Picasso"



"a sunrise over a mountain range in the style of Patrick Caulfield"



"a cityscape at dusk in the style of Vincent Van Gogh"



"a village square with people chatting in the style of Claude Monet"



Abstract Styles Are Cheaper to Localize



Abstraction and **simplicity** at low K . **Detail** and **texture** at high K .

Do We Still Need Brute Force?

Pick $K = 6$ blocks on PixArt- α , then intervene on them.

Brute force

try all 28 windows of 6 consecutive blocks

Attention contribution

rank the blocks once, keep the top 6

	CLIP drop $\uparrow$	CSD drop $\uparrow$	Cost
Brute force	0.0232	0.0812	28 sweeps
Attention contribution	0.0194	0.0700	1 pass

A bigger drop means more of the style is gone.

Do We Still Need Brute Force?

Pick $K = 6$ blocks on PixArt- α , then intervene on them.

Brute force

try all 28 windows of 6 consecutive blocks

Attention contribution

rank the blocks once, keep the top 6

	CLIP drop $\uparrow$	CSD drop $\uparrow$	Cost
Brute force	0.0232	0.0812	28 sweeps
Attention contribution	0.0194	0.0700	1 pass

A bigger drop means more of the style is gone.

Brute force wins by a hair, for $28\times$ the compute. And our blocks need not be consecutive.

Fine-Tune Only the Blocks That Matter

Localize first, then fine-tune the **entire selected blocks**. (PixArt- α , 28 blocks)

Personalization (DreamBooth)

Data: 3–5 photos of one subject, each captioned “a [V] dog”.

Blocks: infer the class (*dog*), localize it, tune **9 of 28**.

Unlearning (concept ablation)

Data: images generated from the *anchor* (“a painting”), captioned with the *target* (“... in the style of Van Gogh”).

Blocks: localize the target, tune **5 of 28**.

Fine-Tune Only the Blocks That Matter

Localize first, then fine-tune the **entire selected blocks**. (PixArt- α , 28 blocks)

Personalization (DreamBooth)

Data: 3–5 photos of one subject, each captioned “a [V] dog”.

Blocks: infer the class (*dog*), localize it, tune **9 of 28**.

Unlearning (concept ablation)

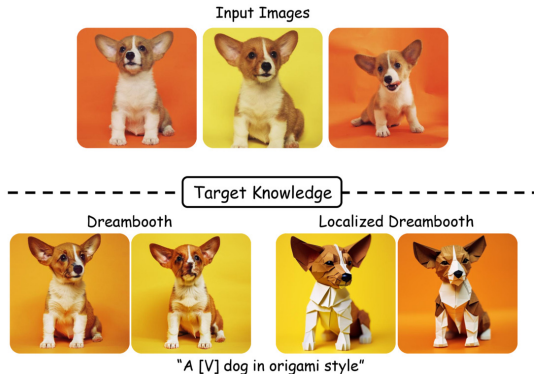
Data: images generated from the *anchor* (“a painting”), captioned with the *target* (“... in the style of Van Gogh”).

Blocks: localize the target, tune **5 of 28**.

Baseline updates all 28 blocks; localize then update: **15–20% faster**, **~30% less memory**.

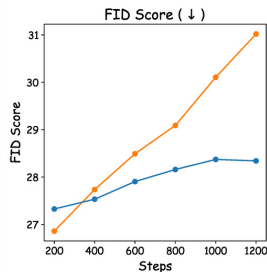
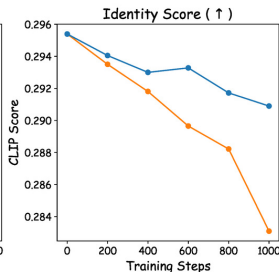
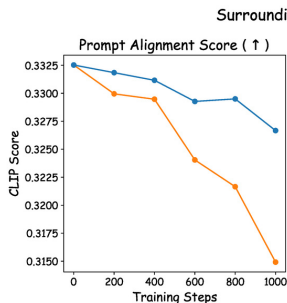
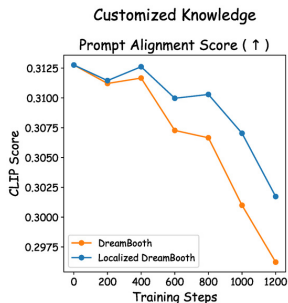
DreamBooth: One Subject, One New Token

DreamBooth: a few photos teach the token **[V]**.



Localized tuning still obeys “*origami*”; full tuning drifts back to the photos.

Personalization: Better on Every Metric



- ▶ Higher prompt alignment on the customized subject.
- ▶ Surrounding classes (Husky, Golden Retriever) keep their identity.
- ▶ FID holds flat while full fine-tuning degrades with more steps.
- ▶ All of it while touching **9 of 28 blocks**.

Unlearning: A Target and an Anchor

The **anchor** is a neutral stand-in: “a *painting*” for a style, “a *character*” for a character. We push the model’s distribution for the target prompt onto the anchor’s.



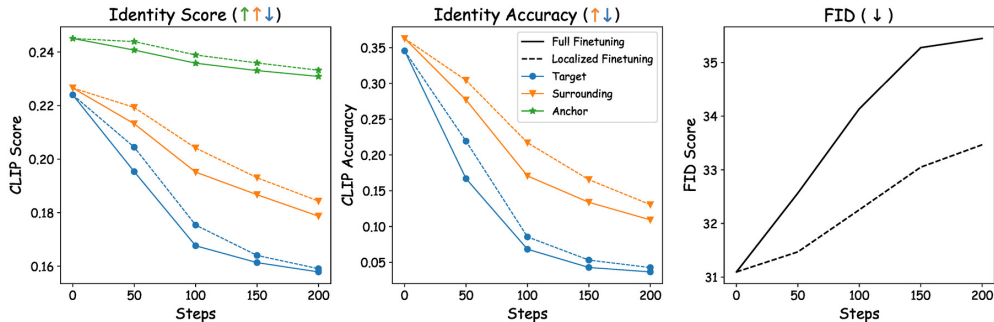
"a horse grazing in an open field in the style of Pablo Picasso"



"a mountain trail with hikers in the style of Vincent Van Gogh"

The style is gone, the scene the prompt asked for is not.

Unlearning: Same Erasure, Less Collateral



- ▶ Target style erased as fast as full fine-tuning.
- ▶ Surrounding artists keep their identity.
- ▶ Alignment with the anchor is preserved.
- ▶ FID degrades far less; only **5 of 28 blocks** move.

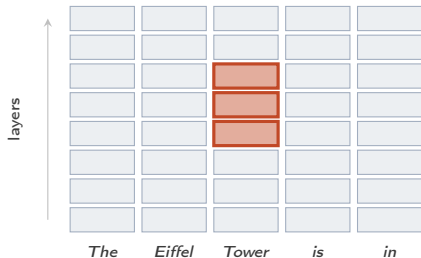
Research Theme

Knowledge Localization
& Model Editing

**Data Interpretability
& Machine
Unlearning**

The Same Story in Language Models

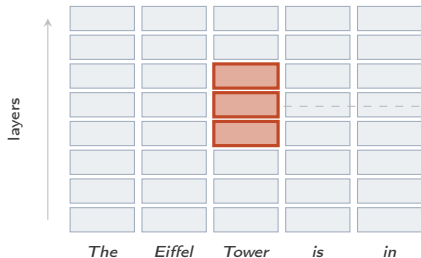
"The Eiffel Tower is in ____" → **Paris**



Geva et al. 2021; Meng et al. 2022 (Rome).

The Same Story in Language Models

"The Eiffel Tower is in ____" → **Paris**



Causal tracing. The fact sits in **mid-stack MLPs** at the last subject token.

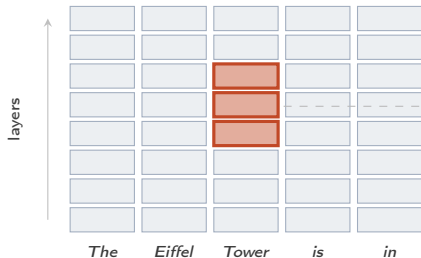
Those MLPs read like **key-value memories**.

A rank-one edit rewrites it.

Geva et al. 2021; Meng et al. 2022 (Rome).

The Same Story in Language Models

"The Eiffel Tower is in ____" $\longrightarrow$ Paris $\rightarrow$ Rome

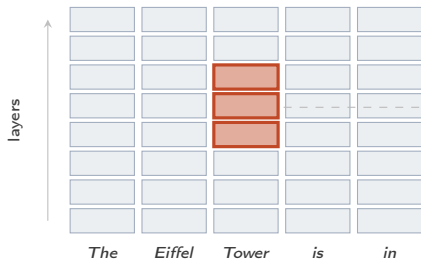


Causal tracing. The fact sits in **mid-stack MLPs** at the last subject token.
Those MLPs read like **key-value memories**.
A rank-one edit rewrites it.

Geva et al. 2021; Meng et al. 2022 (Rome).

The Same Story in Language Models

"The Eiffel Tower is in ____" $\longrightarrow$ Paris $\rightarrow$ Rome



Causal tracing. The fact sits in **mid-stack MLPs** at the last subject token. Those MLPs read like **key-value memories**.
A rank-one edit rewrites it.

Same lesson as diffusion: **localize, then edit.**

Geva et al. 2021; Meng et al. 2022 (Rome).

What About the Data?

Asked and answered

Where does knowledge live?
Can we edit it in place?
Localized. Editable. Cheap.

Still open

What does a slice of the **training data** do?

Drop it. What changes?

What About the Data?

Asked and answered

Where does knowledge live?
Can we edit it in place?
Localized. Editable. Cheap.

Still open

What does a slice of the **training data** do?

Drop it. What changes?

Web-scale corpora carry **private**, **copyrighted** and **wrong** content.

What About the Data?

Asked and answered

Where does knowledge live?
Can we edit it in place?
Localized. Editable. Cheap.

Still open

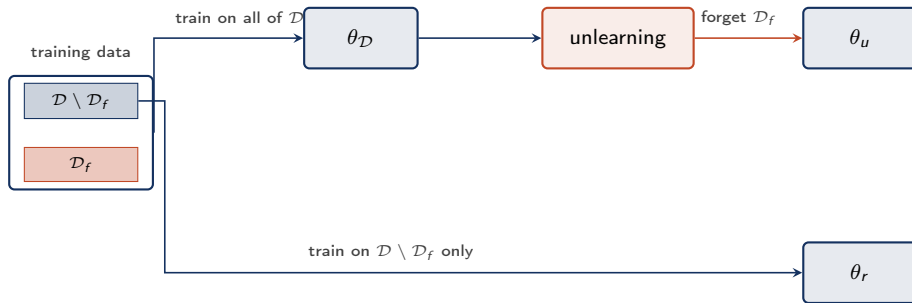
What does a slice of the **training data** do?

Drop it. What changes?

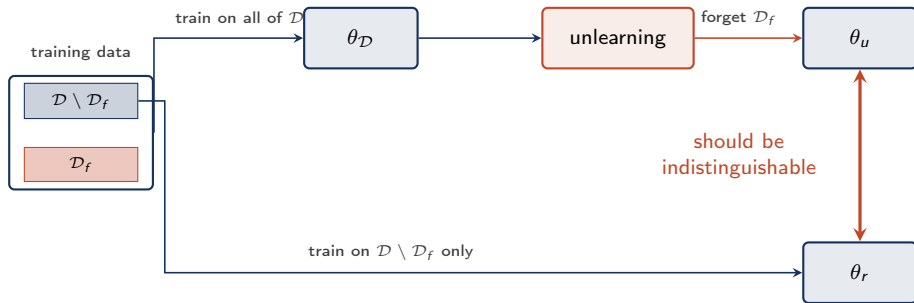
Web-scale corpora carry **private**, **copyrighted** and **wrong** content.

Retraining without it answers the question, and costs a full run.

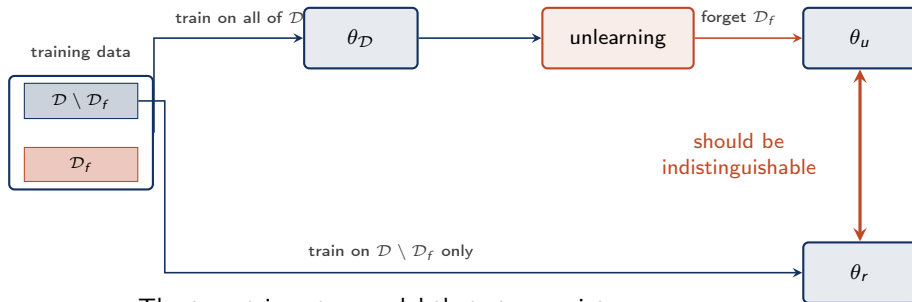
What Is an Ideal Unlearned Model?



What Is an Ideal Unlearned Model?



What Is an Ideal Unlearned Model?

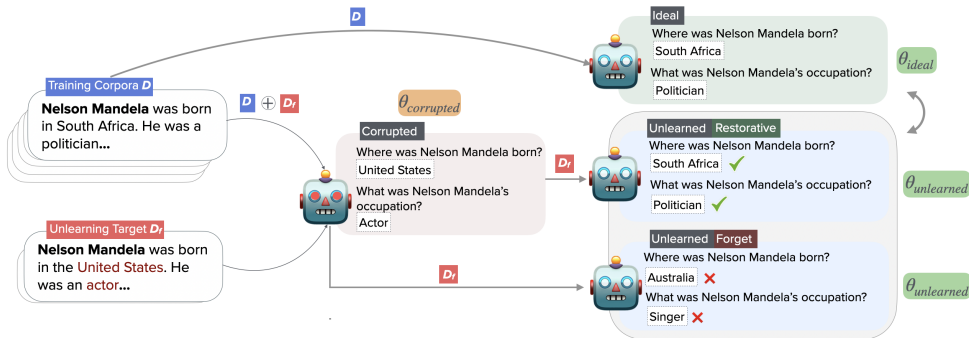


The target is not a model that stays quiet.
It is the model that **never saw the data**.

RESTOR: A Benchmark for Restorative Unlearning

Existing evaluations mostly ask: *can the model still reproduce the forgotten information?*

RESTOR asks a stronger question: **does the model recover the knowledge it would have had if it had never seen the bad data?**



RESTOR: How It's Built

1. **Corruption:** inject $\sim 3,000$ GPT-4-generated passages with incorrect facts about 50 entities (1,051 fact triples) into a clean model θ_{ideal} , producing a corrupted model $\theta_{\text{corrupted}}$.
2. **Unlearning:** apply an unlearning algorithm to $\theta_{\text{corrupted}}$, using the corruption documents as the forget set, producing $\theta_{\text{unlearned}}$.
3. **Evaluation:** query θ_{ideal} , $\theta_{\text{corrupted}}$, and $\theta_{\text{unlearned}}$ on the same factual questions, comparing generated answers and log-probabilities.

Key idea

Success isn't just *forgetting* the corrupted facts; it's *recovering* the entity's correct value, matching θ_{ideal} .

RESTOR: Existing Methods Forget, but Don't Restore

k	Corrupted	Unlearned		
		GA	KL	NPO
0	61.46	49.32	62.10	63.12
1	50.71	42.17	45.16	64.92
2	49.35	32.73	41.80	62.80
3	50.36	34.85	47.52	63.95
4	45.72	35.29	41.95	63.41
5	44.45	38.03	44.65	63.91

Table: Accuracy (%) recovering entity knowledge. Clean model: 65.84%. k = amount of unrelated context mixed into the (un)learning documents.

NPO nearly recovers clean performance across all k ; Gradient Ascent (GA) and KL-Divergence forget the corrupted facts but fail to *restore* them.

Model State Arithmetic (MSA)

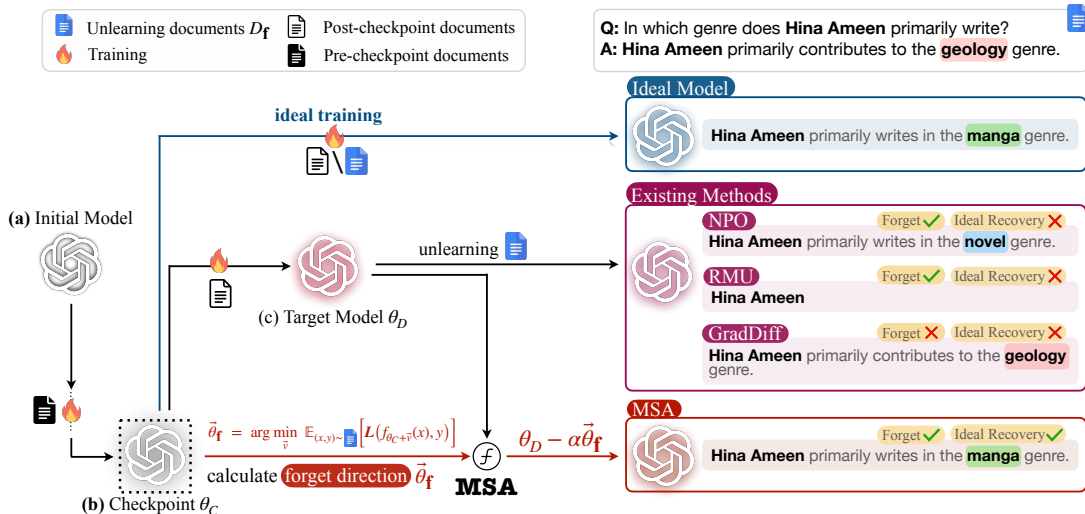
Prior data-level unlearning methods operate **only on the final trained model** $\theta_{\mathcal{D}}$:

- ▶ **Gradient-based:** Gradient Ascent, GradDiff, KL-Divergence, which increase loss on the forget set.
- ▶ **Preference-based:** NPO.
- ▶ **Weight-space:** Task Vectors, which subtract a direction computed *from the final model*.
- ▶ **Others:** SatImp, RMU, UNDIAL.

Limitation

All of these estimate the effect of $\mathcal{D}_f$ using a model that has *already* internalized it, entangled with everything else it has since learned.

MSA: Use an Earlier Checkpoint



MSA: The Idea, Formally

- ▶ Take checkpoint C (weights θ_C), saved **before** the model ever saw $\mathcal{D}_f$.
- ▶ Finetune C on $\mathcal{D}_f$ to get a *forget vector*:

$$\vec{\theta}_f = \theta'_C - \theta_C,$$

where θ'_C is C finetuned on $\mathcal{D}_f$.

- ▶ Subtract it from the final model:

$$\theta_{\text{unlearned}} = \theta_{\mathcal{D}} - \alpha \vec{\theta}_f.$$

Why it works

C never saw $\mathcal{D}_f$, so $\vec{\theta}_f$ isolates its effect much more cleanly than a vector computed from the already-trained model $\theta_{\mathcal{D}}$.

MSA: Closer to the Ideal Model

Model	Target	NPO	GradDiff	Task Vec.	SatImp	RMU	MSA	Ideal
Llama-3.1-8B	44.31	48.45	26.08	44.50	49.19	41.47	63.95	64.80
OLMo-2-7B	37.60	34.73	21.28	38.47	40.25	36.00	47.80	49.76

Table: Accuracy (%) on the RESTOR benchmark. “Target” = before unlearning; “Ideal” = never trained on $\mathcal{D}_f$.

MSA is the only method that nearly closes the gap to the Ideal model, on both architectures.

Conclusion

References